

flight-recorder

scripts/flight-recorder/flight-recorder.sh — out-of-scope host-session flight recorder

Revision: 1 **Last modified:** 2026-08-21T16:05:00Z **Status:** active **Item:** BOB-121 (External watchdog for the forced-logout architectural gap)

Overview

`flight-recorder.sh` appends one structured JSONL sample per minute describing the health of the operator's user session, so that if `user@1000.service` is torn down again — the forced-logout class recorded in BOB-116 / BOB-120 / BOB-123 / BOB-124 / BOB-125 / BOB-126 — the aftermath is diagnosable by reading **one file** instead of being reconstructed from `journalctl` over hours.

It is a **recorder, not a controller**. It observes and writes. It never sends a signal to any process, never kills, never restarts, and never touches host power state (CONST-033). There is deliberately no code path in it that can terminate anything — an assertion its own `verify` subcommand proves against its source, not against a claim in this document.

Why it exists — the architectural gap

BOB-121 names

BOB-126 closed the *cause* of the seven forced-logout incidents (a test's `AsyncMock().pid` defaulting to 1, turning `os.killpg(1, SIGKILL)` into `kill(-1, SIGKILL)`), and a pre-build gate now refuses unguarded `killpg` calls.

BOB-121 is the *remaining* gap: **every monitor the project had lived inside the scope it was monitoring**. The BOB-116 resource-pressure `systemd --user timer` runs inside `user@1000.service`, so when that unit is SIGKILLED the timer dies in the same cascade and records nothing. Incident #3 proved it: the timer fired at 22:42 and 22:57, was next due at ~23:42, and never ran — the kill landed at 23:45:49 and nothing owned by UID 1000 existed again until the operator logged back in at 23:49:00.

At the time this recorder was written, **nothing was watching at all**:

```
$ systemctl show boba-user1000-watchdog.service -p LoadState -p
ActiveState
LoadState=not-found
ActiveState=inactive
```

```
$ systemctl --user show boba-resource-pressure-check.timer -p
ActiveState
ActiveState=inactive
```

The design, and why cron rather than a systemd unit

The load-bearing insight is that **the recorder does not need to survive the event as a process — its *scheduler* does**.

A per-minute tick is a fresh, short-lived process. If a tick were running at the instant of a teardown and died, that costs one sample. What matters is that something still schedules the *next* tick sixty seconds later, inside the dead window, before the operator logs back in. On this host that scheduler is `cron`d, and it is structurally outside the blast radius:

```
$ systemctl show crond -p Slice -p MainPID -p ActiveState
ActiveState=active
MainPID=1531
Slice=system.slice
```

```
$ cat /proc/1531/cgroup
0::/system.slice/crond.service
```

crond has run continuously through every incident. `system.slice` is a peer of `user.slice`, not a descendant, so a `cgroup-subtree kill` of `user@1000.service` cannot reach it.

Correction to the BOB-121 proposal's stated rationale

`docs/proposals/external-watchdog-for-forced-logout-architectural-gap.md` recommended this option (Option B) but predicted that a cron job's process would stay under `crond`'s own process group in `system.slice`, and flagged the live check as a **blocking prerequisite** (its §8 "Phase 1.5"). That check has now been run, and **the prediction was wrong while the conclusion still holds** — for a different and more precise reason.

Measured on this host (ALT Linux, `vixie-cron`, `systemd` 258):

```
--- /proc/self/cgroup ---
0::/user.slice/user-1000.slice/session-15.scope
--- ancestry (pid ppid comm) ---
  pid=2920944 ppid=2920942 comm=(bash)  cgroup=0::/user.slice/
user-1000.slice/session-15.scope
  pid=2920942 ppid=1531    comm=(crond) cgroup=0::/user.slice/
user-1000.slice/session-15.scope
  pid=1531    ppid=1      comm=(crond) cgroup=0::/system.slice/
crond.service
XDG_SESSION_ID=15 XDG_RUNTIME_DIR=/run/user/1000
```

This cron build **does** invoke `pam_systemd`: the job got a session id and was moved into `/user.slice/user-1000.slice/session-15.scope`. So the job is *not* in `system.slice`.

It is nevertheless outside the blast radius, because `session-N.scope` is a **sibling** of `user@1000.service`, not a descendant of it:

```
$ ls -d /sys/fs/cgroup/user.slice/user-1000.slice/*/
/sys/fs/cgroup/user.slice/user-1000.slice/session-5.scope/
/sys/fs/cgroup/user.slice/user-1000.slice/session-6.scope/
/sys/fs/cgroup/user.slice/user-1000.slice/user@1000.service/
```

and the real incident's kill was scoped precisely to `user@1000.service`. All 73 Killing process lines in `docs/qa/BOB-120/journalctl_23-42_to_23-46.log` are attributed to one unit, and the operator's own session scope was *deactivated*, never killed:

```
$ grep -oE '[a-zA-Z0-9@_.\-]+\.(service|scope|slice): Killing
process' \
    docs/qa/B0B-120/journalctl_23-42_to_23-46.log | sort | uniq
-c
73 user@1000.service: Killing process
```

```
$ grep -E 'session-[0-9]+\.(scope)' docs/qa/B0B-120/
journalctl_23-42_to_23-46.log
23:45:49 systemd[1]: session-18.scope: Deactivated successfully.
```

Honest boundary. This makes the recorder survive *the mechanism actually observed in all seven incidents* — a user@1000.service-scoped cascade. It does **not** make it survive a sweep of the whole user-1000.slice, a loginctl terminate-user, or a KillUserProcesses=yes logout path that reaps session scopes. Against those, only the root-owned system.slice unit in scripts/system-slice-watchdog/ is structurally safe. The two are complements, not alternatives: this recorder is the layer that needs **no root and is available today**; that watchdog is the deeper layer and still needs the operator’s one-time su install.

What it captures, and why each signal

Each tick appends one JSON object. The fields exist to answer the project’s own CONST-033 triage protocol *continuously*, instead of reconstructing it later.

Field	Why
ts_utc, epoch, uptime_sec	Uptime continuity — triage step 1. A discontinuity distinguishes a real suspend from a perceived one.
boot_id, boot_changed	Separates a reboot from a session teardown . Same boot id + restarted manager = forced-logout class, definitively.
gap_sec, gap_anomaly	The recording gap is the primary signal. A hole in a per-minute log brackets the dead window to within 60s.
unit_active, unit_sub, unit_start_mono, unit_restarted, unit_down	The teardown fingerprint. A changed non-zero start timestamp means a new instance of the user manager;

Field	Why
	unit_down marks the tick where it was absent. Tracked separately so one event is never counted as two.
unit_pids, unit_pids_peak	Recursive cgroup pid count. Collapse toward zero is the cascade-kill signature.
unit_mem_bytes, unit_mem_peak_bytes	Per-slice memory attribution — was the user slice the thing under pressure?
mem_avail_kb, swap_free_kb, psi_*	The pressure ramp <i>leading into</i> the event, which no post-hoc capture can recover. PSI is sampled because averages hide burst throttling (§11.4.225).
threads_self_uid	The §12.12 thread-exhaustion axis — the 2026-07-07 incident’s mechanism, orthogonal to memory.
k_suspend, k_oom, oom_cgroups, k_unit_kill	Triage steps 2–3, sampled per tick from the journal since the previous tick’s cursor. oom_cgroups carries the attribution that decides the verdict: a libpod-* cgroup means a container hit its own limit; a user@<uid> cgroup means a user-slice OOM perceived as a logout.
journal_ok	Distinguishes “queried, found none” from “could not query”. When false, the counts above are null, never 0 (§11.4.6).

Usage

```
scripts/flight-recorder/flight-recorder.sh tick      # one sample
               (the cron entry)
scripts/flight-recorder/flight-recorder.sh tick-v    # same, echo
               the record
scripts/flight-recorder/flight-recorder.sh report    # one-step
               diagnosis
```

```
scripts/flight-recorder/flight-recorder.sh status      # recorder
health
scripts/flight-recorder/flight-recorder.sh verify      # self-check
preconditions
```

Install / remove the per-minute tick (**no sudo/su required** — this is the reason cron was chosen for this layer):

```
scripts/flight-recorder/install.sh --dry-run          # show the crontab
change
scripts/flight-recorder/install.sh                    # apply it
scripts/flight-recorder/uninstall.sh                  # remove it, keep
the journal
scripts/flight-recorder/uninstall.sh --purge          # remove it and
delete the journal
```

install.sh runs verify first and **refuses to install if it fails** — a scheduled recorder that cannot record is worse than none, because it looks like coverage. Both scripts edit only their own marker-delimited block and back up the previous crontab.

Outputs

Everything lives under \$BOBA_FR_DIR (default \$XDG_STATE_HOME/boba-flight-recorder, i.e. ~/.local/state/boba-flight-recorder):

File	Purpose
journal.jsonl	Append-only sample history, size-bounded with one rotation generation
journal.jsonl.1	Previous rotation generation
latest.json	Last sample, replaced atomically (temp → fsync → rename) so a reader after an abrupt teardown always sees a whole record
state.env	Tick-to-tick continuity: previous epoch, journal cursor, last non-zero unit start, last active state
tick.err	Stderr of the most recent tick only (truncated each run, so it cannot grow)
crontab.backup.<ts>	Pre-change crontab snapshots from install/uninstall

The state dir must be durable. The recorder refuses to run on tmpfs or ramfs, because recording onto them would destroy the evidence in exactly the event it exists to document. This is a hard refusal, not a warning:

```
$ BOBA_FR_DIR=/tmp/x scripts/flight-recorder/flight-recorder.sh
tick
[flight-recorder] ERROR: state dir '/tmp/x' is on tmpfs
(ephemeral). Refusing:
the record would vanish in exactly the event it exists to
document.
$ echo $?
1
```

Environment

Variable	Default	Meaning
BOBA_FR_DIR	\$XDG_STATE_HOME/ boba-flight-recorder	State dir; must be durable
BOBA_FR_TARGET	user@1000.service	Unit to watch Manager owning the target: system or user.
BOBA_FR_SCOPE	system	user exists only to drill the teardown path against a disposable unit
BOBA_FR_TICK_SEC	60	Expected tick spacing; the gap alarm is 2.5× this
BOBA_FR_MAX_BYTES	8388608	Rotate above this size; total is bounded at ~2×
BOBA_FR_TIMEOUT	15	Per-external-command timeout

Edge cases

- **journalctl unreadable** — journal_ok records false and the four journal counters record null. They never record 0, because a blind instrument and a clean host return the same quiet zero (§11.4.201(6)).

- **State file lost** — the next tick records `gap_sec: null` and resumes. No crash, no false gap.
- **Unit stopped, then started later** — the last *non-zero* start timestamp is carried across the intervening ticks, so the new instance is still detected after a dead window in which the unit reported 0.
- **Cron scheduler stopped / host asleep** — produces a gap with no teardown evidence, and report says so explicitly rather than calling it an incident.
- **Single late tick** — jitter under the $2.5\times$ threshold is not flagged; a false alarm is as much a defect as a missed one (§11.4.201(1)).

Anti-bluff verification

`scripts/flight-recorder/flight-recorder.sh verify`

asserts four load-bearing conditions **against the host and against its own source**, not against this document: state dir durability, the scheduler being outside `user.slice`, the absence of any signal-sending call in its own source, and journal readability.

The no-signal assertion is proven non-decorative by a paired §1.1 mutation — injecting one `kill -9` line into a copy makes `verify` FAIL while the unmutated script PASSES. The journal-extraction patterns are proven by a golden-good fixture: run against the real `docs/qa/B0B-120/journalctl_23-42_to_23-46.log` they match the actual SIGKILL line once, and against a benign log they match zero times.

Honest boundaries

- It **cannot prevent** a `kill(-1)`. It makes the aftermath legible; that is all it claims.
- **Untested against a real forced logout.** Survival across a genuine `user@1000.service` teardown is an inference from the cgroup topology and the incident journal, not an observation — no such event has occurred since it was installed, and one cannot be staged safely.
- The staged analogue exercised the teardown-detection code path against a disposable `systemctl --user unit`, not against `user@1000.service`.
- A tick that happens to be mid-write during a teardown can lose that one line; `latest.json` covers the last complete record.

- One-minute resolution. Sub-minute ordering within the event is not recoverable from this log — that is what the `system.slice` watchdog's journal-window capture is for.
- It records ps-free data only, so unlike the `system.slice` watchdog it does **not** capture process cmdlines and therefore carries no §11.4.10 credential exposure in its output.

Related

- `scripts/system-slice-watchdog/` — root-owned `system.slice` watchdog for deep per-incident forensics and SIGKILL initiator attribution. Complementary; still requires the operator's one-time `su` install.
- `docs/guides/forced-logout-flight-recorder.md` — operator guide.
- `docs/proposals/external-watchdog-for-forced-logout-architectural-gap.md` — the BOB-121 design proposal this implements, with the Phase 1.5 correction above.
- `docs/incidents/2026-08-18-3rd-forced-logout.md` and siblings — incident record.